

# Concurrencia y Paralelismo

Grado en Ingeniería Informática

Julio 2020

## 1. Futures [0.75 puntos]

Una future es una abstracción para retrasar computaciones. Imaginemos que queremos calcular  $c = f(a)$  pero que  $c$  no lo usamos hasta mucho más adelante en el programa. Podemos transformar esto como `future = promise(f, a)` y en el uso de  $c$  como `result = force(future)`. Se da una implementación de referencia de `promise` y `force`. Puede asumirse que:

- Las funciones que vamos a usar toman un argumento `void *` y devuelven un valor del mismo tipo.
- Se puede hacer más de un `force` para cada `future`. La operación `free_future` libera la `future`.

Implementar `future` para que cree un thread para realizar la operación y `force` para que si el thread ya ha terminado devuelva el valor calculado y en caso contrario espere a que el thread termine. La función `free_future` libera la memoria usada por la `future`. Pueden añadirse todos los campos que se consideren necesarios a la `struct future`. Notese que `free_future` tiene que liberar también la memoria del resultado.

Ejemplo de uso:

---

```
struct arg {
    int i;
    int result;
}

void *add_one(void *v)
{
    struct arg *arg = v;
    arg->result = arg->i + 1;
    return arg->result;
}

int main(void)
{
    struct arg a;
    struct future *future;
    int *result;

    a.i = 5;
    future = promise(add_one, &a);
    result = force(future);
    printf("The result is %d\n", *result);
    free_future(future);
    return 0;
}
```

---

```
struct future {
    void *(*f)(void*);
    void *arg;
    void * result;
};

struct future *promise(void *(*f)(void *), void *arg)
{
    struct future *future = malloc(sizeof(struct future));
    assert(future)
    return NULL;
}
```

```
        future->f = f;
        future->arg = arg;
        return future;
    }

void *force(struct future *future)
{
    assert(future);
    future->result = future->f(future->arg);
    return future->result;
}

void free_future(struct future *future)
{
    assert(future);
    free(result);
    free(future);
}



---


```

## Solución

---

```
struct future {
    void *(*f)(void*);
    void *arg;
    bool done;
    pthread_cond_t cond;
    pthread_mutex_t mutex;
};

void *calculate(void *arg)
{
    struct future *future = arg;
    void *result;
    result = future->f(future->arg);
    pthread_mutex_lock(&future->mutex);
    future->result = result;
    future->done = true;
    pthread_mutex_unlock(&future->mutex);
    pthread_cond_signal(&future->cond);

    return NULL;
}

struct future *promise(void *(*f)(void *), void *arg)
{
    struct future *future = malloc(sizeof(struct future));
    pthread_t thr;
    if (!future)
        return NULL;
    future->f = f;
    future->arg = arg;
    future->done = false;
    pthread_create(&thr, NULL, calculate, future);
    return future;
}

void force(struct future *future)
{
    void *result;
    assert(future);

    pthread_mutex_lock(&future->mutex);
    while(!future->done)
        pthread_cond_wait(&future->cond, &future->mutex);
    pthread_mutex_unlock(&future->mutex);
}

void free_future(struct future *future)
{
    void *result;
    assert(future);

    pthread_mutex_lock(&future->mutex);
    while(!future->done)
        pthread_cond_wait(&future->cond, &future->mutex);
    pthread_mutex_unlock(&future->mutex);
    free(future);
}
```

---

## 2. Paso de peatones [0.75 puntos]

Vamos a modelar un paso de peatones, donde cada peatón y cada coche está representado por un thread. El cruce no tiene semáforo, por lo que los peatones tienen prioridad. Varios peatones pueden cruzar el paso simultáneamente, pero la carretera tiene un único carril y los coches tienen que pasar de uno a uno en el orden en que llegan al cruce.

---

```
struct crossing {
    pthread_mutex_t *lock;
    int pedestrians;           // Number of pedestrians (waiting or crossing)
    int cars_crossing;         // Number of cars crossing
    pthread_cond_t pedestrian_wait;
    queue *car_queue;
};

int queue_size(queue *);
pthread_cond_t *queue_remove(queue *);
void queue_insert(queue *, pthread_cond_t *);

void *pedestrian(void *ptr) {
    struct crossing *cross = ptr;

    pthread_mutex_lock(cross->lock);
    cross->pedestrians++;
    while(cross->cars_crossing > 0)
        pthread_cond_wait(cross->pedestrian_wait, cross->lock);
    pthread_mutex_unlock(cross->lock);

    cross();

    pthread_mutex_lock(cross->lock);
    cross->pedestrians--;
    if(cross->pedestrians==0 && queue_size(cross->car_queue) >0) {
        cross->cars_crossing++;
        pthread_cond_t *c = queue_remove(cross->car_queue);
        pthread_cond_signal(c);
    }
    pthread_mutex_unlock(cross->lock);

    return NULL;
}

void *car(void *ptr) {
    struct crossing *cross = ptr;
    ...
    cross();
    ...
}
```

---

Implemente la función car con el comportamiento del coche. Puede añadir más campos a la estructura crossing si lo considera necesario.

## Solución

---

```
struct crossing {
    pthread_mutex_t *lock;
    int pedestrians;
    int cars_crossing;
    pthread_cond_t pedestrian_wait;
    queue *car_queue;
};

int queue_size(queue *);
pthread_cond_t *queue_remove(queue *);
void queue_insert(queue *, pthread_cond_t *);

void *pedestrian(void *ptr) {
    struct crossing *cross = ptr;

    pthread_mutex_lock(cross->lock);
    cross->pedestrians++;
    while(cross->cars_crossing > 0)
        pthread_cond_wait(cross->pedestrian_wait, cross->lock);
    pthread_mutex_unlock(cross->lock);

    cross();

    pthread_mutex_lock(cross->lock);
    cross->pedestrians--;
    if(cross->pedestrians==0 && queue_size(cross->car_queue) >0) {
        cross->cars_crossing++;
        pthread_cond_t *c = queue_remove(cross->car_queue);
        pthread_cond_signal(c);
    }
    pthread_mutex_unlock(cross->lock);
}

void *car(void *ptr) {
    struct crossing *cross = ptr;

    pthread_mutex_lock(cross->lock);
    if(cross->pedestrians > 0 || cross->cars_crossing > 0) {
        pthread_cond_t c;
        pthread_cond_init(&c, NULL);
        queue_insert(cross->q, &c);
        pthread_cond_wait(&c, cross->lock);
    } else cross->cars_crossing++;
    pthread_mutex_unlock(cross->lock);

    cross();

    pthread_mutex_lock(cross->lock);
    cross->cars_crossing--;
    if(cross->pedestrians > 0)
        pthread_cond_broadcast(cross->pedestrian_wait);
    else if(queue_size(cross->car_queue)>0) {
        cars_crossing++;
        pthread_cond_t *c = queue_remove(cross->car_queue);
        pthread_cond_signal(c);
    }
    pthread_mutex_unlock(cross->lock);
}
```

---

### 3. Árbol de Procesos [0.5 puntos]

Tenemos un sistema donde hay un árbol binario de procesos, donde cada nodo es un proceso que guarda un valor y dos pids correspondientes a los subárboles izquierdo y derecho. Los nodos se crean llamando a `start_node/1`, y añadimos hijos a un nodo existente con `add_left_child/2` y `add_right_child/2`.

---

```
-module(tree).

-export([start_node/1, add_left_child/2, add_right_child/2, height/1, init_node/1]).

%% API

start_node(V) ->
    spawn(?MODULE, init_node, [V]).

add_left_child(Tree, Child_Tree) ->
    Tree ! {add_left_child, Child_Tree}.

add_right_child(Tree, Child_Tree) ->
    Tree ! {add_right_child, Child_Tree}.

get_value(Tree) ->
    ...

exists(Tree, V) ->
    ...

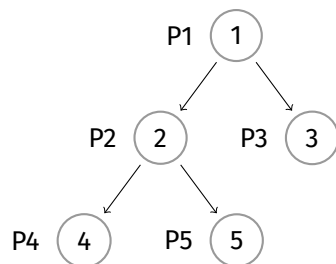
%% Internal functions

init_node(V) ->
    node_loop(V, none, none).

node_loop(V, Left, Right) ->
    receive
        {add_left_child, Child_Tree} ->
            node_loop(V, Child_Tree, Right);
        {add_right_child, Child_Tree} ->
            node_loop(V, Left, Child_Tree);
        ...
    end.
```

---

Por ejemplo, el árbol:



Se crearía de la siguiente forma:

---

```
P1 = start_node(1).
P2 = start_node(2).
P3 = start_node(3).
P4 = start_node(4).
P5 = start_node(5).
add_left_child(P2, P4).
add_right_child(P2, P5).
add_left_child(P1, P2).
add_right_child(P1, P3).
```

---

Implemente la función `get_value/1` que devuelve el valor de la raíz de un árbol, y la función `exists/2`, que devuelve `true` si un valor existe en el árbol, y `false` si no existe.

---

```

-module(tree).

-export([start_node/1, add_left_child/2, add_right_child/2, height/1, init_node/1]).

%% API

start_node(V) ->
    spawn(?MODULE, init_node, [V]).

add_left_child(Tree, Child_Tree) ->
    Tree ! {add_left_child, Child_Tree}.

add_right_child(Tree, Child_Tree) ->
    Tree ! {add_right_child, Child_Tree}.

get_value(Tree) ->
    Tree ! {get_value, self()},
    receive
        {get_value_reply, V} ->
            V
    end.

exists(Tree, V) ->
    Tree ! {exists, V, self()},
    receive
        {exists_reply, R} ->
            R
    end.

%% Internal functions

init_node(V) ->
    node_loop(V, none, none).

node_loop(V, Left, Right) ->
    receive
        {add_left_child, Child_Tree} ->
            node_loop(V, Child_Tree, Right);
        {add_right_child, Child_Tree} ->
            node_loop(V, Left, Child_Tree);
        {get_value, From} ->
            From ! {get_value_reply, V},
            node_loop(V, Left, Right);
        {exists, V, From} ->
            From ! {exists_reply, true},
            node_loop(V, Left, Right);
        {exists, Val, From} ->
            From ! {exists_reply, check_exists(Left, Val) or check_exists(Right, Val)},
            node_loop(V, Left, Right)
    end.

check_exists(none, _) -> false;
check_exists(Tree, V) -> exists(Tree, V).

```

---

# EXAMEN DE CONCURRENCIA Y PARALELISMO

## Bloque II: Paralelismo

APELLIDOS: \_\_\_\_\_ NOMBRE: \_\_\_\_\_

Convocatoria extraordinaria

10 de julio de 2020

**IMPORTANTE:** Al comienzo del documento de respuesta, cada alumno debe indicar su nombre completo y número del documento de identidad o pasaporte.

De cara a contestar las preguntas que figuran a continuación, los datos concretos serán diferentes para cada alumno, dependiendo de su número de DNI (pasaporte para alumnos sin nacionalidad española). Considera tu DNI como  $C_1C_2C_3C_4C_5C_6C_7C_8L$  (en caso de alumnos sin nacionalidad española,  $C_1C_2C_3C_4C_5C_6C_7$  son las siete cifras que aparecen en el número de pasaporte).

- [1p] 1. **Conceptos de paralelismo.** Disponemos de un algoritmo que tiene como estado inicial una matriz  $S$  bidimensional de  $M$  filas y  $N$  columnas de tipo entero. Iterativamente, el algoritmo genera nuevos estados a partir del propio contenido de la matriz. El estado siguiente ( $k+1$ ) para cada una de las celdas de la matriz se calcula mediante una función  $f$  que tiene como parámetros el valor de esa celda y las celdas colindantes del estado actual ( $k$ ), según la siguiente fórmula:

$$S_{i,j}^{k+1} = f(S_{i-1,j}^k, S_{i,j-1}^k, S_{i,j+1}^k, S_{i+1,j}^k)$$

Las posiciones con índices que coinciden fuera de rango de la matriz se corresponden con las posiciones del extremo opuesto. Por ejemplo,  $S_{-1,j}^k = S_{M-1,j}^k$ , y  $S_{i,N}^k = S_{i,0}^k$ .

El siguiente pseudocódigo resume el funcionamiento del algoritmo implementado, donde  $S0$  es el estado actual,  $S^k$ , y  $S1$  es el estado siguiente,  $S^{k+1}$ . El algoritmo itera hasta que no se producen cambios entre dos estados consecutivos. La función  $f$  se encapsula dentro de otra función,  $evoluciona(S^k, i, j)$ , que recibe como parámetros el estado actual y los índices de la posición de la matriz que queremos calcular.

```
1  S1 = estado_inicial
2  S0 = estado_vacio
3
4  while (S1 != S0)
5  {
6      S0 = S1
7      for (i = 0..M-1)
8          for (j = 0..N-1)
9              S1[i,j] = evoluciona(S0, i, j)
10 }
```

Queremos paralelizar el algoritmo para ejecutarlo en un sistema dedicado en exclusiva a resolver este problema, que cuenta con 16 procesadores. El coste de ejecutar una vez la función *evoluciona* es de  $T_f$  ns. Por las características de la red de interconexión, realizar una comunicación entre procesos tiene un coste fijo de  $2^{20}$  ns, y adicionalmente tarda  $T_c$  ns por cada elemento de la matriz enviado. Es decir, un envío de  $k$  elementos tardará  $2^{20} + kT_c$  ns. Las variables  $M$ ,  $N$ ,  $T_f$  y  $T_c$  adoptan los siguientes valores:



- $M = 2^{C_1+C_2+4}$
- $N = 2^{C_3+C_4+4}$
- $T_f = 2^{C_5}$  ns
- $T_c = 2^{C_6}$  ns

Para este ejercicio ignoraremos el coste de las comunicaciones necesarias para repartir el estado inicial entre los procesos.

Responde justificadamente las siguientes preguntas. Una respuesta sin razonamiento correcto no será válida aunque el resultado o conclusión sean correctos.

- ¿Qué tipo de descomposición debemos aplicar al problema? ¿Sería eficiente utilizar una estrategia de asignación dinámica centralizada? [0,1p]
- Para una asignación estática por bloques, consideramos los 16 procesos estructurados lógicamente en mallas bidimensionales de 16x1, 8x2 y 4x4. Para cada una de estas tres configuraciones, indica cuántas comunicaciones y de cuántos elementos debe efectuar cada proceso en cada iteración del algoritmo. [0,3p]
- Teniendo en cuenta los tiempos mencionados en el enunciado ( $T_c$  y  $T_f$ ), ¿qué configuración del apartado anterior es la más adecuada? [0,3p]
- Si distribuimos los procesos en malla bidimensional de 4x4, ¿cuál será el speedup esperado? [0,3p]

- [1p] 2. **Diseño de algoritmos paralelos.** Hemos paralelizado el algoritmo descrito en el ejercicio anterior para una distribución de tareas estática por **bloques de filas**.

El tamaño de la matriz ( $M \times N$ ) es el indicado en el ejercicio anterior. En esta ocasión, el algoritmo paralelo diseñado asume también un número constante de procesadores  $P=2^{C_1+2}$ .

El proceso 0 se encarga de leer el estado inicial y a continuación ese estado inicial se reparte entre todos los procesos. Como  $M$ ,  $N$  y  $P$  son valores constantes, podemos calcular estáticamente el tamaño de las matrices locales que corresponden a cada proceso:  $M_{local} \times N_{local}$ .  $M_{local}$  y  $N_{local}$  también serán constantes en nuestro código.

El pseudocódigo de la paralelización es el siguiente:

```
... // inicializacion de MPI

if (rango == 0) {
    lee_estado_inicial(S_inicial, M, N)
}
reparte_estado_inicial(S_inicial, S1)

S0 = estado_vacio

while (continua_bucle(S1, S0))
{
    S0 = S1
    intercambia_datos(S0, rango)

    for (i = 1 .. M_local )
        for (j = 0 .. N_local-1 )
            S1[i,j] = evoluciona(S0, i, j)
}
```

Como es una distribución por bloques de filas, para el estado local de cada proceso reservamos 2 filas adicionales,  $M_{local} + 2$  filas en total, para recoger las filas de los procesos vecinos. De esa

forma, las filas propias del proceso se localizarán entre los índices 1 y  $M_{local}$ , la fila 0 queda reservada para recibir la última fila del proceso superior, y la fila  $M_{local} + 1$  se reserva para recibir la primera fila del proceso inferior.

El bucle principal debe reorganizarse para realizar el trabajo local en cada proceso. Además de los índices del bucle for, se necesitan 2 modificaciones. En primer lugar, en la condición de salida del bucle no es suficiente con que cada proceso compruebe de manera independiente si sus estados S1 y S0 son iguales. El bucle debe terminar si y solo si los estados S1 y S0 son iguales en **todos** los procesos. Para ello contamos con una función denominada “**continua\_bucle**”, que devolverá 0 (false) si las matrices S1 y S0 son iguales en todos los procesos, o 1 (true) en caso contrario. En segundo lugar, en el bucle es necesario establecer comunicaciones entre los procesos vecinos para enviar la primera y última filas, que se colocarán en las filas adicionales reservadas en los estados locales. Esta tarea está contenida dentro de la función “**intercambia\_datos**”.

- (a) Determina los valores de  $M_{local}$  y  $N_{local}$ . [0,1p]
- (b) Proporciona una implementación concreta en lenguaje C de la función “intercambia\_datos”. Dentro de la función puedes hacer uso de las constantes descritas en el enunciado del ejercicio. [0,45p]
- (c) Proporciona una implementación concreta en lenguaje C de la función “continua\_bucle”. Dentro de la función puedes hacer uso de las constantes descritas en el enunciado del ejercicio. [0,45p]

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype datatype, MPI_Op op,
               int root, MPI_Comm comm)
```

## SOLUCIÓN:

### 1. Ejercicio 1

- (a) Debemos aplicar una descomposición de dominio. El coste fijo de establecer una comunicación es muy alto ( $2^{20}$  ns), y en una asignación dinámica centralizada (maestro-esclavo), en cada iteración el proceso maestro debe gestionar las comunicaciones con los procesos esclavos secuencialmente. Como la evaluación de la función *evoluciona* tiene un coste fijo y la carga puede balancearse bien, las comunicaciones generarían un cuello de botella. Por lo tanto, una estrategia de asignación dinámica no sería necesaria ni eficiente.
- (b) 16x1 Es una distribución por bloques de filas. Cada proceso mantiene una matriz de  $\frac{M}{16} \times N$  elementos. Cada proceso realiza 2 envíos, a los procesos superior e inferior, cada uno de ellos de  $N$  elementos.
- 8x2 Cada proceso mantiene una matriz de  $\frac{M}{8} \times \frac{N}{2}$  elementos. Cada proceso realiza 3 envíos. En vertical debe enviar  $\frac{N}{2}$  elementos a los procesos superior e inferior, y en horizontal enviará  $\frac{2M}{8}$  elementos al otro proceso de la misma fila. También se puede considerar que en horizontal son necesarios 2 envíos al mismo proceso en lugar de uno solo (4 en total), de  $\frac{M}{8}$  elementos cada uno de ellos.
- 4x4 Cada proceso mantiene una matriz de  $\frac{M}{4} \times \frac{N}{4}$  elementos. Cada proceso realiza 4 envíos, a los procesos colindantes en las 4 direcciones. En vertical debe enviar  $\frac{N}{4}$  elementos en cada mensaje a los procesos superior e inferior, y en horizontal enviará  $\frac{M}{4}$  elementos en cada mensaje a los procesos a su izquierda y derecha.
- (c) La respuesta a este apartado dependerá de los valores de  $M$ ,  $N$ ,  $T_c$ . El tiempo de ejecutar la función,  $T_f$ , no influye porque la carga computacional de cada proceso es la misma independientemente de la configuración de los procesos. Para cada una de las configuraciones debemos calcular el tiempo empleado en las comunicaciones por iteración (en nanosegundos) de la siguiente forma:

$$T_{comm} = n\_envios \times 2^{20} + n\_elementos \times T_c$$

Donde  $n\_envios$  es el número de envíos que hace cada proceso y  $n\_elementos$ , el número total de elementos que envían. Entonces:

$$T_{comm}(16x1) = 2 \times 2^{20} + 2NT_c$$

$$T_{comm}(8x2) = 3 \times 2^{20} + \left(\frac{2M}{8} + 2\frac{N}{2}\right)T_c$$

$$T_{comm}(4x4) = 4 \times 2^{20} + \left(2\frac{M}{4} + 2\frac{N}{4}\right)T_c$$

Seleccionaremos la distribución de procesos que implique el menor tiempo de comunicación.

- (d) En función de los valores de las variables, la aceleración resultado variará desde valores muy próximos a cero hasta valores cercanos a 16 (aceleración lineal). La carga computacional de cada proceso es siempre  $\frac{MN}{16}$ , porque está perfectamente balanceada, y por lo tanto el tiempo de computación es  $T_f \frac{MN}{16}$ .

$$t_{seq} = T_f MN$$

$$t_{par} = T_{comm}(4x4) + T_f \frac{MN}{16}$$

$$speedup = \frac{t_{seq}}{t_{par}}$$

## 2. Ejercicio 2

- (a)  $M_{local} = \frac{M}{P} = \frac{M}{2^{C_1+2}}$ ,  $N_{local} = N$
- (b) Para declarar la función podemos definir un estado  $S$  como “int \*” o “int \*\*”. Como el intercambio se produce por filas individuales, es indiferente si las filas se encuentran reservadas consecutivamente en memoria o no. Por tanto, hemos elegido “int \*\*” por comodidad.

Si la matriz  $S$  estuviese declarada como “int \*”, la dirección de la fila  $i$  de la matriz en lugar de ser  $S[i]$  sería  $\&(S[i*N_{local}])$ , o bien  $S + i*N_{local}$ .

```
void intercambia_datos(int **S, rango)
{
    int inferior = (rango + 1) % P;
    int superior = (rango + P - 1)%P;

    if (rango % 2)
    {
        MPI_Send(S[1], N, MPI_INT, superior, 0, MPI_COMM_WORLD);
        MPI_Send(S[M_local], N, MPI_INT, inferior, 0, MPI_COMM_WORLD);

        MPI_Recv(S[M_local+1], N, MPI_INT, inferior, 0, MPI_COMM_WORLD);
        MPI_Recv(S[0], N, MPI_INT, superior, 0, MPI_COMM_WORLD);
    }
    else
    {
        MPI_Recv(S[M_local+1], N, MPI_INT, inferior, 0, MPI_COMM_WORLD);
        MPI_Recv(S[0], N, MPI_INT, superior, 0, MPI_COMM_WORLD);

        MPI_Send(S[1], N, MPI_INT, superior, 0, MPI_COMM_WORLD);
        MPI_Send(S[M_local], N, MPI_INT, inferior, 0, MPI_COMM_WORLD);
    }
}
```

- (c) Igual que en el ejercicio anterior, hemos escogido declarar las matrices  $S1$  y  $S0$  como “int \*\*”. Cada proceso debe comprobar si existen diferencias entre sus estados  $S1$  y  $S0$ , y a continuación se hace una reducción de tipo LOR (OR lógico) para verificar si existen diferencias entre los estados locales de alguno de los procesos, y un broadcast para que la función retorne el mismo valor en todos los procesos. Dado que la variable *distintas* solo toma valores 0 o 1, también funcionaría correctamente la operación BOR (bitwise OR), y además, como C interpreta cualquier valor distinto de 0 como “true”, también es válida una reducción de tipo suma.

```
int continua_bucle(int **S1, int **S0)
{
    int distintas = 0;
    int debe_continuar;

    for (int i=1; i <= M_local; i++)
        for (int j=0; j < M_local; j++)
            if (S1[i][j] != S0[i][j])
            {
                distintas = 1;
                break;
            }

    MPI_Reduce(&distintas, &debe_continuar, 1, MPI_INT,
              MPI_LOR, 0, MPI_COMM_WORLD);
    MPI_Bcast(&debe_continuar, 1, MPI_INT, 0, MPI_COMM_WORLD);

    return debe_continuar;
}
```